Technical Design Document

GitHub: https://github.com/ashucfx/isocode-cardealership
Live application: Not deployed; local run instructions are in ‘deployment.md‘.

Problem and scope

Brand X needs a premium customer journey that converts browsing into a test-drive lead and gives consultants an actionable inbox. This build covers inventory, transparent Lucknow pricing, test-drive booking, lead assignment, consultant assistance, and admin pricing control. Payments, DMS, WhatsApp, service booking, and analytics are future scope.

Assumptions

- Lucknow is the initial city and has two fictional showrooms.
- Admin is the authorized used-car pricing role.
- A local JSON repository is used for zero-dependency evaluation; the route and domain boundaries are designed for PostgreSQL/Prisma replacement. This is an explicit take-home trade-off, not a production database recommendation.
- AI output is assistance only and must be reviewed.

Architecture

The Node server exposes REST-like JSON routes and serves a responsive static client. Domain operations (price calculation, booking conflict checks, assignment and authorization) execute server-side. The client renders API data and never owns authoritative state.

Business rules

‘onRoad = exShowroom + roadTax + insurance + accessories‘. Only active, non-expired city records participate. Test-drive uniqueness is enforced by showroom/date/slot. A new lead is assigned to a consultant at the selected showroom; an unassigned lead is never silently presented as owned.

Security and privacy

Role checks run on every protected endpoint. Session cookies are HttpOnly and SameSite. Booking requires explicit consent, and registration numbers are masked before leaving the server. Production requires hashed passwords, HTTPS, rate limiting, CSRF protection and a managed secret store.

AI

The server constructs prompts from the lead and vehicle records only. In a deployed environment, ‘AI_PROVIDER_URL‘ and ‘AI_PROVIDER_API_KEY‘ connect this boundary to the configured server-side LLM provider; keys never reach the browser. Summary and reply are separate, labelled actions; drafts are editable and never sent automatically. The local demo uses a deterministic development adapter so reviewers can exercise the complete UI without a third-party key. In production, provider output must be schema-validated and rejected on malformed or unsupported content. No AI output affects price, inventory or booking availability.

Testing and performance

‘npm test‘ covers the deterministic price calculation and missing-pricing behavior. The booking route re-checks availability to prevent races at the application boundary. Production follow-up should add PostgreSQL transaction tests and Playwright coverage for the buyer-to-consultant journey. Static assets are small and images use responsive remote URLs; performance must be measured on the deployment target before making a 4G claim.